

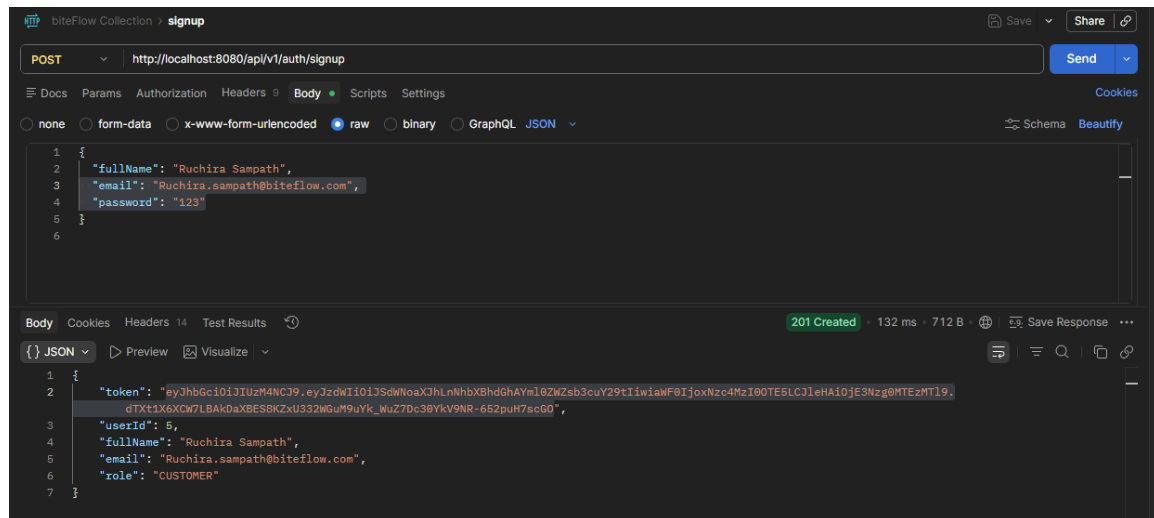
BiteFlow API Documentation

Complete API reference for the Spring Boot BiteFlow Food Ordering System backend.

1. Authentication APIs

1. Register User

HTTP Method	POST
Endpoint	/auth/signup
Description	Registers a new customer account and returns a JWT token.
Authentication	Public
Full URL	http://localhost:8080/api/v1/auth/signup



2. Login User

HTTP Method	POST
Endpoint	/auth/signin
Description	Authenticates a user and returns a JWT token.
Authentication	Public
Full URL	http://localhost:8080/api/v1/auth/signin

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/v1/auth/signin
- Body (raw):**

```
1 {
2   "email": "Ruchira.sampath@biteflow.com",
3   "password": "123"
4 }
5
```
- Status:** 200 OK
- Response Time:** 249 ms
- Response Size:** 707 B
- Body (JSON):**

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTQwIiwiaWF0Ij01OTIwMzZab3cuY29tIiwiaWF0Ij0xNzI1MjQ0LjE1eHA10jE3Nzg8MTE2NDR9",
3   "userId": 5,
4   "fullName": "Ruchira Sampath",
5   "email": "Ruchira.sampath@biteflow.com",
6   "role": "CUSTOMER"
7 }
```

2. Category APIs

1. Get All Categories

HTTP Method	GET
Endpoint	/categories
Description	Retrieves all food categories.
Authentication	Public
Full URL	http://localhost:8080/api/v1/categories

The screenshot displays a REST client interface with the following details:

- Collection:** biteFlow Collection
- Item Name:** get all categories
- Method:** GET
- URL:** http://localhost:8080/api/v1/categories
- Auth Type:** Inherit auth from parent
- Response Status:** 200 OK
- Response Time:** 34 ms
- Response Size:** 1.08 KB
- Response Body (JSON):**

```
[{"id": 5, "name": "Pizza", "description": "Delicious Italian pizzas with various toppings"}, {"id": 6, "name": "Burgers", "description": "Juicy burgers with premium ingredients"}, {"id": 7, "name": "Pasta", "description": "Traditional Italian pasta dishes"}, {"id": 8, "name": "Desserts", "description": "Sweet treats and desserts"}, {"id": 9, "name": "Beverages", "description": "Refreshing drinks and beverages"}]
```

2. Create Category

HTTP Method	POST
Endpoint	/categories
Description	Creates a new food category.
Authentication	ADMIN
Full URL	http://localhost:8080/api/v1/categories

The screenshot displays a REST client interface with the following details:

- Collection:** biteFlow Collection > Category APIs > create category
- Method:** POST
- URL:** http://localhost:8080/api/v1/categories
- Body (Request):**

```
1 {
2   "name": "Rice Dishes",
3   "description": "Fried rice, biryani, and other rice specialties"
4 }
5
6
```
- Status:** 201 Created (128 ms, 514 B)
- Body (Response):**

```
1 {
2   "id": 14,
3   "name": "Rice Dishes",
4   "description": "Fried rice, biryani, and other rice specialties"
5 }
```
- Console:** Shows the POST request to http://localhost:8080/api/v1/auth/sign...

3. Update Category

HTTP Method	PUT
Endpoint	/categories/{categoryId}
Description	Updates an existing category.
Authentication	ADMIN
Full URL	http://localhost:8080/api/v1/categories/{categoryId}

The screenshot shows a REST client interface with the following details:

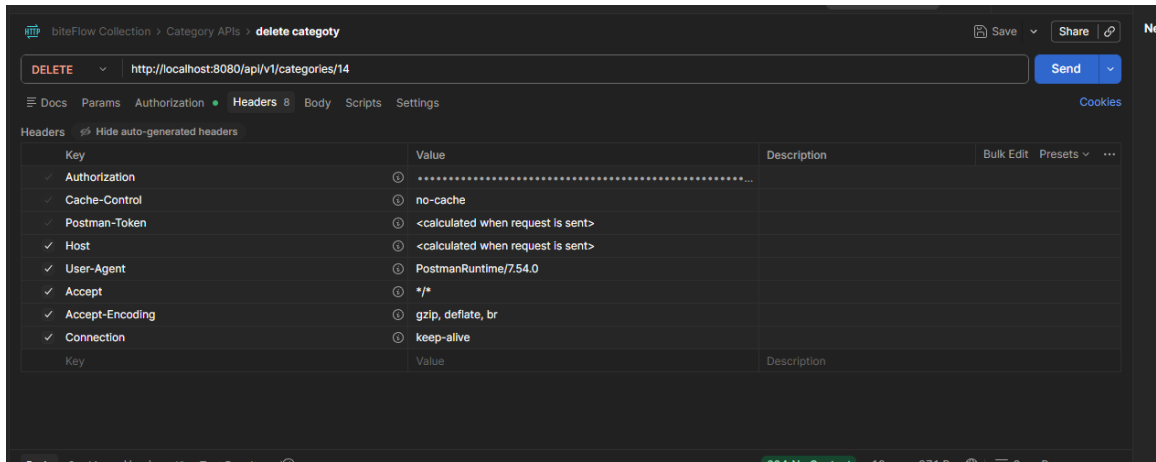
- Collection:** biteFlow Collection > Category APIs > update category
- Method:** PUT
- URL:** http://localhost:8080/api/v1/categories/14
- Body (Request):**

```
1 {
2   "name": "Rice Dishes",
3   "description": "Fried rice, Biryani, Rice & Curry and other rice specialties"
4 }
5
```
- Status:** 200 OK, 49 ms, 523 B
- Body (Response):**

```
1 {
2   "id": 14,
3   "name": "Rice Dishes",
4   "description": "Fried rice, Biryani, Rice & Curry and other rice specialties"
5 }
```

4. Delete Category

HTTP Method	DELETE
Endpoint	/categories/{categoryId}
Description	Deletes an existing category.
Authentication	ADMIN
Full URL	http://localhost:8080/api/v1/categories/{categoryId}



3. Food APIs

1. Get All Food Items

HTTP Method	GET
Endpoint	/foods
Description	Retrieves all food items.
Authentication	Public
Full URL	http://localhost:8080/api/v1/foods

biteFlow Collection > Food APIs > get all food items

GET http://localhost:8080/api/v1/foods

Send

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 14 Test Results

200 OK · 27 ms · 3.16 KB

JSON Preview Visualize

```
1 [
2   {
3     "id": 39,
4     "name": "Tuna Sandwich",
5     "description": "Fresh tuna salad sandwich",
6     "price": 100.0,
7     "stockQuantity": 40,
8     "status": "AVAILABLE",
9     "categoryId": 8,
10    "categoryName": "Desserts"
11  }
```

2. Get Available Food Items

HTTP Method	GET
Endpoint	/foods?availableOnly=true
Description	Retrieves available food items only.
Authentication	Public
Full URL	http://localhost:8080/api/v1/foods?availableOnly=true

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8080/api/v1/foods?availableOnly=true
- Query Params:** availableOnly=true
- Status:** 200 OK
- Response Body (JSON):**

```
[
  {
    "id": 39,
    "name": "Tuna Sandwich",
    "description": "Fresh tuna salad sandwich",
    "price": 100.0,
    "stockQuantity": 40,
    "status": "AVAILABLE",
    "categoryId": 8,
    "categoryName": "Desserts"
  }
]
```


3. Create Food Item

HTTP Method	POST
Endpoint	/foods
Description	Creates a new food item.
Authentication	ADMIN
Full URL	http://localhost:8080/api/v1/foods

The screenshot shows a REST client interface with the following details:

- Collection:** biteFlow Collection > Food APIs > create food item
- Method:** POST
- URL:** http://localhost:8080/api/v1/foods
- Body (raw):**

```
1 {
2   "name": "Chicken Burger",
3   "description": "Burger with cheese",
4   "price": 1500.0,
5   "stockQuantity": 20,
6   "status": "AVAILABLE",
7   "categoryId": 6
8 }
```
- Response (JSON):**

```
1 {
2   "id": 54,
3   "name": "Chicken Burger",
4   "description": "Burger with cheese",
5   "price": 1500.0,
6   "stockQuantity": 20,
7   "status": "AVAILABLE",
8   "categoryId": 6,
9   "categoryName": "Burgers"
10 }
```
- Status:** 201 Created · 110 ms · 584 B
- Console:** X-XSS-Protection: "0"

4. Update Food Item

HTTP Method	PUT
Endpoint	/foods/{foodItemId}
Description	Updates an existing food item.
Authentication	ADMIN
Full URL	http://localhost:8080/api/v1/foods/{foodItemId}

The screenshot displays a REST client interface with the following details:

- Collection:** biteFlow Collection > Food APIs > update food item
- Method:** PUT
- URL:** http://localhost:8080/api/v1/foods/54
- Body (raw):**

```
1 {
2   "id": 54,
3   "name": "Chicken Burger",
4   "description": "Burger with cheese - updated",
5   "price": 1500.0,
6   "stockQuantity": 20,
7   "status": "AVAILABLE",
8   "categoryId": 6,
9   "categoryName": "Burgers"
10 }
```
- Response:** 200 OK - 41 ms - 589 B. The response body is identical to the request body.
- Console:**

```
X-XSS-Protection: "0"
Cache-Control: "no-cache, no-store, max-age=0, must-revalidate"
Pragma: "no-cache"
```

5. Delete Food Item

HTTP Method	DELETE
Endpoint	/foods/{foodItemId}
Description	Deletes a food item.
Authentication	ADMIN
Full URL	http://localhost:8080/api/v1/foods/{foodItemId}

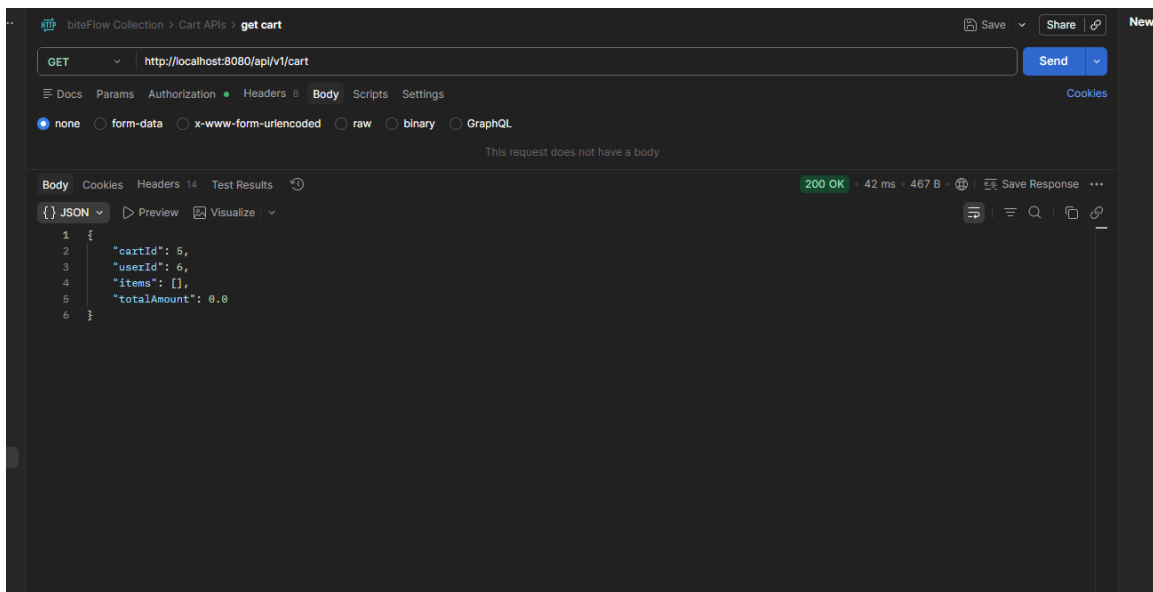
The screenshot shows the Postman interface for a DELETE request. The top bar indicates the collection is 'biteFlow Collection' and the API is 'Food APIs'. The request is named 'delete food item'. The method is 'DELETE' and the URL is 'http://localhost:8080/api/v1/foods/54'. The 'Send' button is visible. The 'Authorization' tab is selected, showing 'Bearer Token' as the auth type. A token is entered in the 'Token' field, masked with dots. Below the token field, a note states: 'The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.' The bottom section shows the 'Body' tab with a '204 No Content' status, a response time of '61 ms', and a size of '371 B'. The 'Raw' tab is selected, showing a single line of response body.

4. Cart APIs

1. Get Cart

HTTP Method	GET
Endpoint	/cart
Description	Retrieves authenticated user's cart.
Authentication	CUSTOMER
Full URL	http://localhost:8080/api/v1/cart

Need to sign in as a customer



2. Add Item To Cart

HTTP Method	POST
Endpoint	/cart/items
Description	Adds a food item to the cart.
Authentication	CUSTOMER
Full URL	http://localhost:8080/api/v1/cart/items

The screenshot shows a REST client interface with the following details:

- Collection:** biteFlow Collection > Cart APIs > add item to the cart
- Method:** POST
- URL:** http://localhost:8080/api/v1/cart/items
- Body (Request):**

```
1 {
2   "foodItemId": 39,
3   "quantity": 2
4 }
5
```
- Response:** 200 OK, 69 ms, 597 B. The response body is JSON:

```
1 {
2   "cartId": 5,
3   "userId": 6,
4   "items": [
5     {
6       "cartItemId": 1,
7       "foodItemId": 39,
8       "foodName": "Tuna Sandwich",
9       "unitPrice": 199.0,
10      "quantity": 2,
11      "lineTotal": 298.0,
12      "stockQuantity": 49
13    }
14  ],
15   "totalAmount": 299.0
16 }
```

3. Update Cart Item

HTTP Method	PUT
Endpoint	/cart/items/{cartItemId}
Description	Updates cart item quantity.
Authentication	CUSTOMER
Full URL	http://localhost:8080/api/v1/cart/items/{cartItemId}

The screenshot shows a REST client interface with the following details:

- Collection:** biteFlow Collection > Cart APIs > update card item
- Method:** PUT
- URL:** http://localhost:8080/api/v1/cart/items/1
- Body (raw):**

```
1 {
2   "quantity": 3
3 }
```
- Response (JSON):**

```
1 {
2   "cartId": 5,
3   "userId": 6,
4   "items": [
5     {
6       "cartItemId": 1,
7       "foodItemId": 39,
8       "foodName": "Tuna Sandwich",
9       "unitPrice": 100.0,
10      "quantity": 3,
11      "lineTotal": 300.0,
12      "stockQuantity": 40
13    }
14  ],
15  "totalAmount": 300.0
16 }
```
- Status:** 200 OK · 24 ms · 597 B

4. Remove Cart Item

HTTP Method	DELETE
Endpoint	/cart/items/{cartItemId}
Description	Removes a cart item.
Authentication	CUSTOMER
Full URL	http://localhost:8080/api/v1/cart/items/{cartItemId}

The screenshot displays a REST client interface with the following details:

- Collection:** biteFlow Collection > Cart APIs > delete cart
- Method:** DELETE
- URL:** http://localhost:8080/api/v1/cart/items/1
- Body:** none (This request does not have a body)
- Response:** 200 OK, 32 ms, 467 B
- JSON Response:**

```
1 {
2   "cartId": 5,
3   "userId": 6,
4   "items": [],
5   "totalAmount": 9.9
6 }
```

5. Clear Cart

HTTP Method	DELETE
Endpoint	/cart
Description	Clears all cart items.
Authentication	CUSTOMER
Full URL	http://localhost:8080/api/v1/cart

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/api/v1/cart`
- Method:** `DELETE`
- Auth Type:** `Bearer Token`
- Token:** `*****`
- Response:** `200 OK` (21 ms, 467 B)
- Body:**

```
{
  "cartId": 5,
  "userId": 6,
  "items": [],
  "totalAmount": 0.0
}
```


5. Order APIs

1. Place Order

HTTP Method	POST
Endpoint	/orders
Description	Places a new customer order.
Authentication	CUSTOMER
Full URL	http://localhost:8080/api/v1/orders

The screenshot displays a REST client interface with the following details:

- Collection:** biteFlow Collection > Order APIs > place order
- Method:** POST
- URL:** http://localhost:8080/api/v1/orders
- Body (raw):**

```
{  "deliveryAddress": "Phone: 1234567890 Address: IJSE, Panadura"}
```
- Response (JSON):**

```
{  "orderId": 1,  "userId": 6,  "customerName": "Test",  "status": "PLACED",  "deliveryAddress": "Phone: 1234567890 Address: IJSE, Panadura",  "totalAmount": 200.0,  "createdAt": "2026-06-09T18:27:34.0687288",  "items": [    {      "orderItemId": 1,      "foodItemId": 39,      "foodName": "Tuna Sandwich",      "quantity": 2,      "unitPrice": 100.0,      "lineTotal": 200.0    }  ],  "paymentStatus": "PENDING"}
```
- Status:** 201 Created
- Time:** 67 ms
- Size:** 755 B
- Console:** X-XSS-Protection: "0"

2. Get Current User Orders

HTTP Method	GET
Endpoint	/orders/me
Description	Retrieves authenticated user's orders.
Authentication	CUSTOMER
Full URL	http://localhost:8080/api/v1/orders/me

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8080/api/v1/orders/me
- Auth Type:** Bearer Token
- Token:** [Redacted]
- Status:** 200 OK
- Response Body (JSON):**

```
[
  {
    "orderId": 1,
    "userId": 6,
    "customerName": "Test",
    "status": "PLACED",
    "deliveryAddress": "Phone: 1234567890 Address: IJSE, Panadura",
    "totalAmount": 200.0,
    "createdAt": "2026-05-09T18:27:34.068729",
    "items": [
      {
        "orderItemId": 1,
        "foodItemId": 39,
        "foodName": "Tuna Sandwich",
        "quantity": 2,
        "unitPrice": 100.0,
        "lineTotal": 200.0
      }
    ],
    "paymentStatus": "PENDING"
  }
]
```
- Console:** X-XSS-Protection: "0"

3. Cancel Own Order

HTTP Method	PATCH
Endpoint	/orders/{orderId}/cancel
Description	Cancels customer's own order.
Authentication	CUSTOMER
Full URL	http://localhost:8080/api/v1/orders/{orderId}/cancel

The screenshot displays the Postman application interface for a REST client. The top bar shows the collection 'biteFlow Collection' and the specific endpoint 'cancel own order'. The HTTP method is set to 'PATCH' and the URL is 'http://localhost:8080/api/v1/orders/1/cancel'. The 'Authorization' tab is active, showing a 'Bearer Token' type with a masked token. The 'Body' tab is selected, displaying a JSON response. The response status is '200 OK' with a response time of 44 ms and a size of 751 B. The JSON body contains order details, including a cancelled status, delivery address, and a list of items.

biteFlow Collection > Order APIs > cancel own order

PATCH http://localhost:8080/api/v1/orders/1/cancel

Auth Type: Bearer Token

Token: [REDACTED]

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body: 200 OK • 44 ms • 751 B

```
{
  "orderId": 1,
  "userId": 6,
  "customerName": "Test",
  "status": "CANCELLED",
  "deliveryAddress": "Phone: 1234567890 Address: IJSE, Panadura",
  "totalAmount": 200.0,
  "createdAt": "2026-05-09T10:27:34.060729",
  "items": [
    {
      "orderItemId": 1,
      "foodItemId": 39,
      "foodName": "Tuna Sandwich",
      "quantity": 2,
      "unitPrice": 100.0,
      "lineTotal": 200.0
    }
  ],
  "paymentStatus": "FAILED"
}
```

4. Get All Orders

HTTP Method	GET
Endpoint	/orders
Description	Retrieves all orders.
Authentication	ADMIN
Full URL	http://localhost:8080/api/v1/orders

biteFlow Collection > Order APIs > ADMIN - get all orders

GET http://localhost:8080/api/v1/orders Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers 14 Test Results 200 OK · 22 ms · 1.06 KB Save Response

JSON Preview Visualize

```
17     "lineTotal": 200.0
18   },
19 ],
20 "paymentStatus": "FAILED"
21 },
22 {
23   "orderId": 2,
24   "userId": 6,
25   "customerName": "Test",
26   "status": "PLACED",
27   "deliveryAddress": "Phone: 1234567890 Address: IJSE, Panadura",
28   "totalAmount": 200.0,
29   "createdAt": "2026-06-09T18:31:54.40828",
30   "items": [
31     {
32       "orderItemId": 2,
33       "foodItemId": 39,
34       "foodName": "Tuna Sandwich",
35       "quantity": 2,
36       "unitPrice": 100.0,
37       "lineTotal": 200.0
38     }
39   ],
40   "paymentStatus": "PENDING"
41 }
42 ]
```

5. Update Order Status

HTTP Method	PATCH
Endpoint	/orders/{orderId}/status
Description	Updates order status.
Authentication	ADMIN
Full URL	http://localhost:8080/api/v1/orders/{orderId}/status

The screenshot shows a REST client interface with the following details:

- Collection:** biteFlow Collection > Order APIs > ADMIN - update order status
- Method:** PATCH
- URL:** http://localhost:8080/api/v1/orders/2/status
- Body:** raw (selected), containing the JSON:

```
{
  "status": "PREPARING"
}
```
- Response:** 200 OK, 45 ms, 751 B. The response body is JSON, showing the updated order details:

```
{
  "orderId": 2,
  "userId": 6,
  "customerName": "Test",
  "status": "PREPARING",
  "deliveryAddress": "Phone: 1234567890 Address: IJSE, Panadura",
  "totalAmount": 280.0,
  "createdAt": "2026-05-09T18:31:54.40828",
  "items": [
    {
      "orderItemId": 2,
      "foodItemId": 39,
      "foodName": "Tuna Sandwich",
      "quantity": 2,
      "unitPrice": 100.0,
      "lineTotal": 200.0
    }
  ],
  "paymentStatus": "PENDING"
}
```

6. Payment APIs

1. Update Payment

HTTP Method	PATCH
Endpoint	/payments
Description	Updates payment information.
Authentication	CUSTOMER
Full URL	http://localhost:8080/api/v1/payments

The screenshot displays a REST client interface for a collection named "biteFlow Collection" under the "Payment APIs" group, specifically for the "update payment" endpoint. The HTTP method is set to "PATCH" and the URL is "http://localhost:8080/api/v1/payments". The request body is formatted as raw JSON, containing the following data:

```
1 {
2   "orderId": 3,
3   "status": "COMPLETED",
4   "transactionReference": "12345"
5 }
6
```

The response status is "200 OK" with a response time of 34 ms and a body size of 549 B. The response body is also in JSON format, containing the following data:

```
1 {
2   "paymentId": 3,
3   "orderId": 3,
4   "amount": 200.0,
5   "status": "COMPLETED",
6   "paidAt": "2026-06-09T18:42:53.8788173",
7   "transactionReference": "12345"
8 }
```

2. Get Payment For Order

HTTP Method	GET
Endpoint	/payments/order/{orderId}
Description	Retrieves payment for a specific order.
Authentication	ADMIN or CUSTOMER
Full URL	http://localhost:8080/api/v1/payments/order/{orderId}

The screenshot shows a REST client interface with a sidebar on the left listing various API collections. The main panel displays a GET request to the endpoint `http://localhost:8080/api/v1/payments/order/3`. The request is successful, returning a 200 OK status with a response time of 20 ms and a body size of 548 B. The response body is a JSON object containing the following data:

```
1 {
2   "paymentId": 3,
3   "orderId": 3,
4   "amount": 200.0,
5   "status": "COMPLETED",
6   "psidat": "2020-08-09T18:42:53.878817",
7   "transactionReference": "12345"
8 }
```

3. Get All Payments

HTTP Method	GET
Endpoint	/payments
Description	Retrieves all payment records.
Authentication	ADMIN
Full URL	http://localhost:8080/api/v1/payments

The screenshot shows a REST client interface with the following details:

- URL:** http://localhost:8080/api/v1/payments
- Method:** GET
- Auth Type:** Bearer Token
- Token:** [Redacted]
- Status:** 200 OK
- Response Time:** 23 ms
- Response Size:** 805 B
- Response Body (JSON):**

```
[{"paidAt": "2026-06-09T18:38:11.666722", "transactionReference": null}, {"paymentId": 2, "orderId": 2, "amount": 200.0, "status": "PENDING", "paidAt": "2026-06-09T18:31:64.413809", "transactionReference": null}, {"paymentId": 3, "orderId": 3, "amount": 200.0, "status": "COMPLETED", "paidAt": "2026-06-09T18:42:53.878817", "transactionReference": "12345"}]
```


7. User APIs

1. Get All Users

HTTP Method	GET
Endpoint	/users
Description	Retrieves all users.
Authentication	ADMIN
Full URL	http://localhost:8080/api/v1/users

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8080/api/v1/users
- Body:** none (selected)
- Status:** 200 OK
- Response Time:** 26 ms
- Response Size:** 912 B
- Response Body (JSON):**

```
[{"id": 5, "fullName": "Ruchira Sampath", "email": "Ruchira.sampath@biteflow.com", "role": "ADMIN"}, {"id": 6, "fullName": "Test", "email": "test@biteflow.com", "role": "CUSTOMER"}]
```

2. Create User

HTTP Method	POST
Endpoint	/users
Description	Creates a new user account.
Authentication	ADMIN
Full URL	http://localhost:8080/api/v1/users

The screenshot displays a REST client interface with the following details:

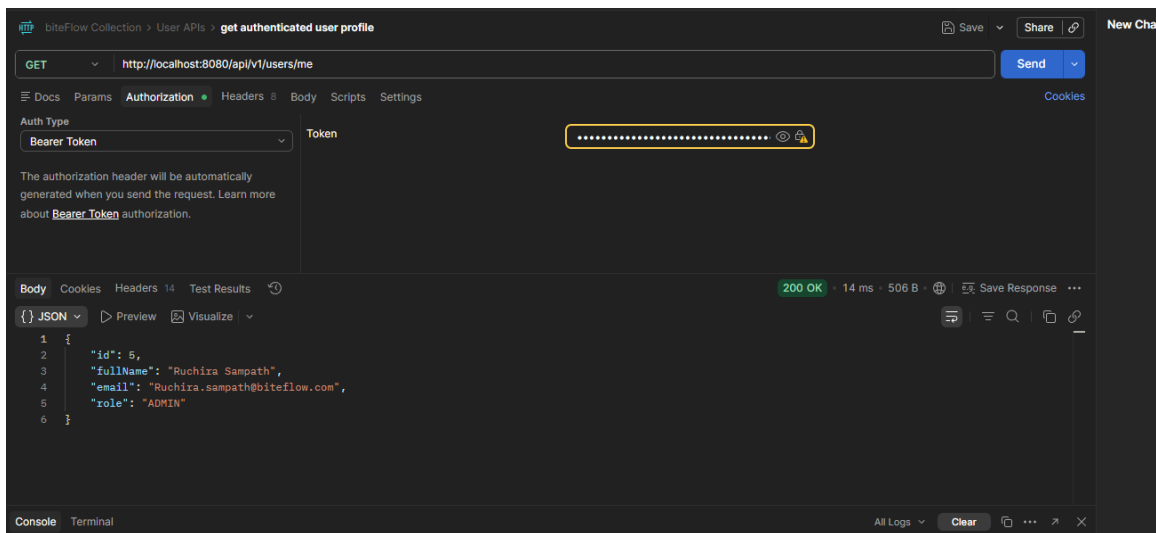
- Request:**
 - Method: POST
 - URL: http://localhost:8080/api/v1/users
 - Body (raw):

```
1 {
2   "fullName": "Admin-biteflow",
3   "email": "adminbiteflow@biteflow.com",
4   "password": "123",
5   "role": "ADMIN"
6 }
```
- Response:**
 - Status: 201 Created
 - Time: 135 ms
 - Size: 508 B
 - Body (JSON):

```
1 {
2   "id": 7,
3   "fullName": "Admin-biteflow",
4   "email": "adminbiteflow@biteflow.com",
5   "role": "ADMIN"
6 }
```

4. Get Current User Profile

HTTP Method	GET
Endpoint	/users/me
Description	Retrieves authenticated user profile.
Authentication	Authenticated User
Full URL	http://localhost:8080/api/v1/users/me



8. Appendix

Business Rules

- Empty carts cannot place orders
- Stock quantity is validated before ordering
- Cancelled orders restore stock
- Delivered orders cannot be modified
- Payments initialize as PENDING
- Customers can only access their own resources
- At least one ADMIN account must remain